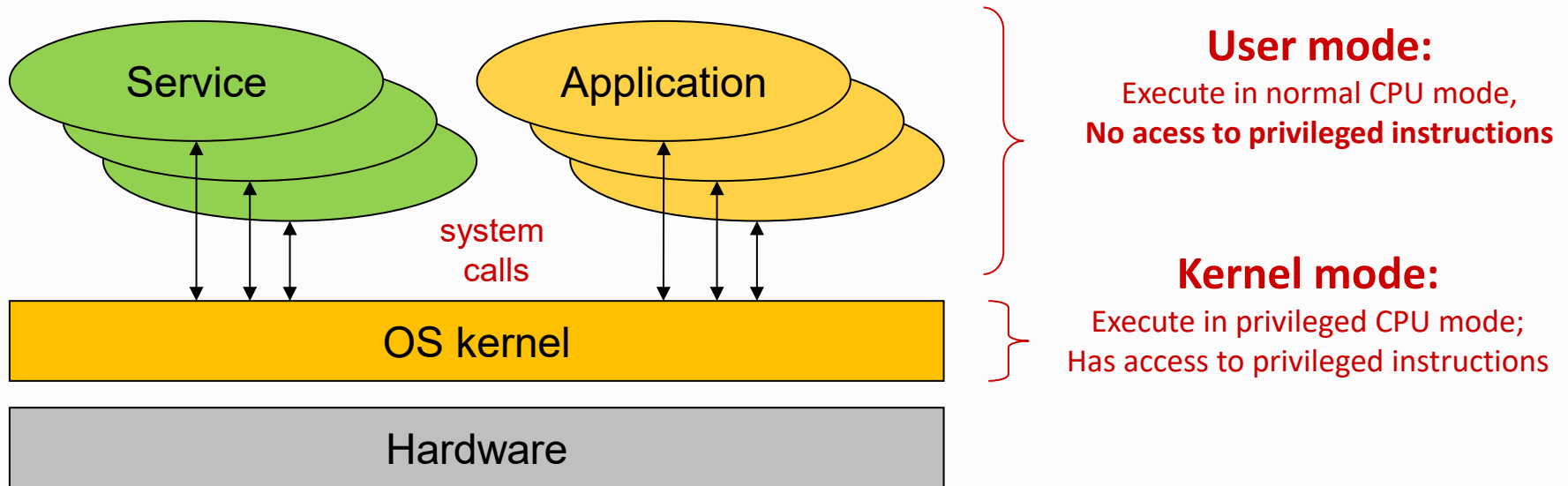


Operating systems

Operating Systems

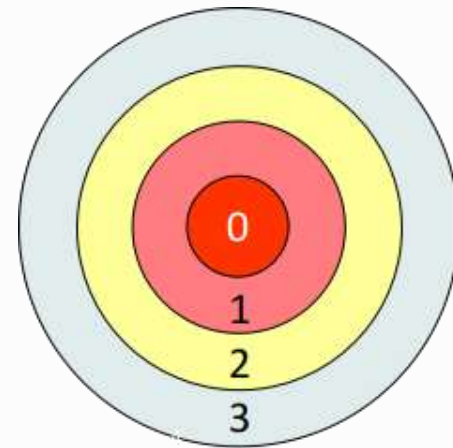


Kernel Objectives

- ▷ Initialize devices (boot time)
- ▷ Virtualize the hardware
 - ♦ Computational model
- ▷ Enforce protection policies and provide protection mechanisms
 - ♦ Against involuntary mistakes
 - ♦ Against non-authorized activities
- ▷ Provide a file system
 - ♦ Agnostic of the actual storage devices used

Execution Rings

- ▷ Different levels of privilege
 - ◆ Forming a set of concentric rings
 - ◆ Used by CPUs to prevent non-privileged code from running privileged opcodes
 - e.g., IN/OUT, TLB manipulation
- ▷ Nowadays processors have 4 rings
 - ◆ But OS's usually use only 2
 - 0 (supervisor/kernel mode)
 - 3 (user-mode)
- ▷ Transfer of control between rings requires special gates
 - ◆ The ones that are used by system calls (aka syscalls)
 - Traps
 - ◆ Interrupt gates



Executing Virtual Machines

▷ Common approach

- ◆ Software-based virtualization
- ◆ Direct execution of guest user-mode code (ring 3)
- ◆ Binary translation of privileged code (ring 0)
 - Guest OS kernels remain unchanged, but do not run directly on the host machine

▷ Hardware-assisted virtualization

- ◆ Full virtualization
 - There is a ring -1 below ring 0
 - Hypervisor and kernel extensions such as Intel VT-x and AMD-V
- ◆ It can virtualize hardware for many ring 0 kernels
 - No need of binary translation
 - Guest OS's run faster (almost native performance)

Execution of Virtual Machines

- ▷ Virtual machines implement an essential security mechanism: confinement
 - ◆ Implement a security domain constrained for use of a small set of applications
 - ◆ Also provide a common abstraction with common hardware
 - Even if the host hardware is modified
- ▷ Provide additional mechanisms
 - ◆ Control resources
 - ◆ Prioritize access to resources
 - ◆ Creation of images for analysis
 - ◆ Fast recovery to a known state

Computational Model

▷ Set of entities (objects) managed by the OS kernel

- ◆ Define how applications interact with the kernel

▷ Examples

- ◆ User identifiers
- ◆ Processes
- ◆ Virtual memory
- ◆ Files and file systems
- ◆ Communication channels
- ◆ Physical devices

- Storage
 - Tapes, magnetic disks, optical disks
 - Silicon disks
- Network interfaces
 - Wired, wireless
- Human-computer interfaces
 - Keyboards, mice
 - Graphical screens, text consoles
- Serial/parallel I/O interfaces
 - USB, Bluetooth
 - Serial ports, parallel ports, infrared

User Identifiers (UID)

- ▶ For the OS kernel a user is a number
 - ◆ Established during a login operation
 - ◆ User ID (UID)
- ▶ All activities are executed on a computer on behalf of a UID
 - ◆ UID allows the kernel to assert what is allowed/denied to them

User Identifiers (UID)

- ▷ Linux: UID 0 is omnipotent (root)
 - ♦ Administration activities are usually executed with UID 0
- ▷ macOS: UID 0 is omnipotent for management
 - ♦ Some binaries and activities are restricted, even for root
- ▷ Windows: concept of privileges
 - ♦ For administration, system configuration, etc.
 - ♦ There is no unique, well-known administrator identifier
 - ♦ Administration privileges can be bound to several UIDs
 - Usually through administration groups
 - Administrators, Power Users, Backup Operators

Group Identifiers (GID)

- ▶ OS also address group identifiers
 - ◆ A group is composed by zero or more users
 - ◆ A group may be composed by other groups
 - ◆ Group ID: Integer value (Linux, Android, macOS) or UUID (Windows)
- ▶ User may belong to multiple groups
 - ◆ User rights = rights of its UID + rights of its GIDs
- ▶ In Linux, activities always execute under the scope of a set of groups
 - ◆ 1 primary group: user to define the ownership of created files
 - ◆ N secondary groups: used to condition access to resources

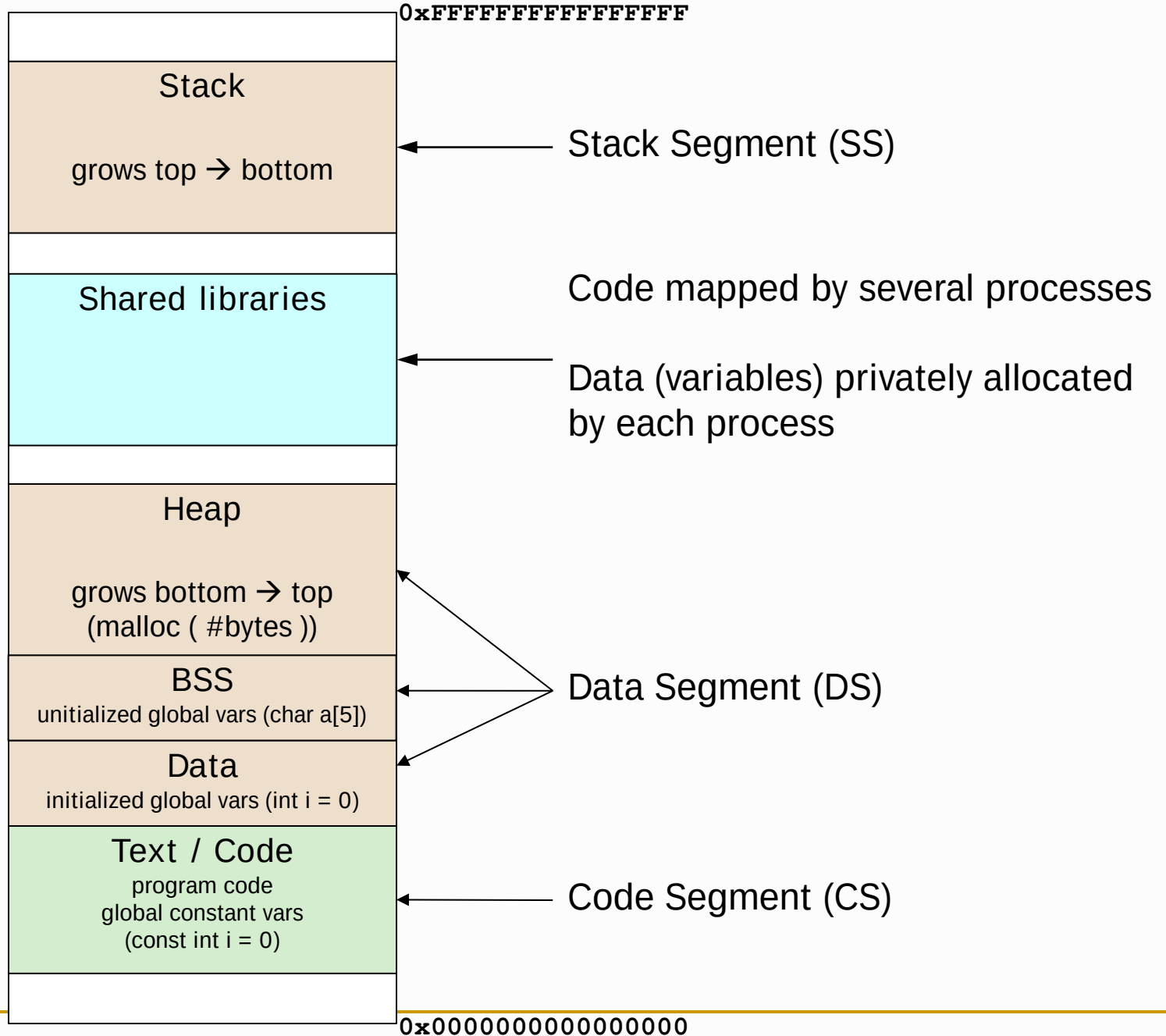
Processes

- ▷ A process defines the context of an activity
 - ♦ For taking security-related decisions
 - ♦ For other purposes (e.g., scheduling)

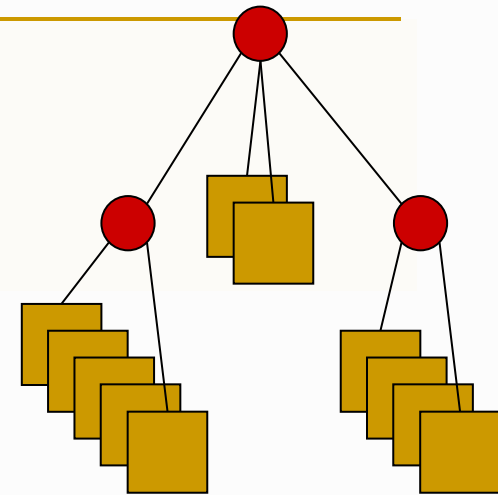
- ▷ Security-related context
 - ♦ Effective Identity (eUID and eGIDs)
 - Fundamental for enforcing access control
 - May be the same as the identity of the user launching the process
 - ♦ Resources being used
 - Open files
 - Including communication channels
 - Reserved virtual memory areas
 - CPU time used, priority, affinity, namespace

Virtual Memory

- ▷ The address space where activities take place
 - ♦ Have the maximum size defined by the hardware architecture
 - 32 bits -> 2^{32} Bytes
 - 64 bits -> 2^{64} Bytes
 - ♦ Managed in small chunks named pages (4 KiB)
- ▷ Virtual Memory can be sparse
 - ♦ Only the pages used must be allocated
 - ♦ Although processes always see a contiguous memory space
- ▷ Virtual Memory is mapped to RAM when actually used
 - ♦ At a given moment, the RAM has pages from multiple address spaces
 - ♦ The choice of how to manage those spaces is very important
 - Avoid fragmentation, management memory according to their freshness

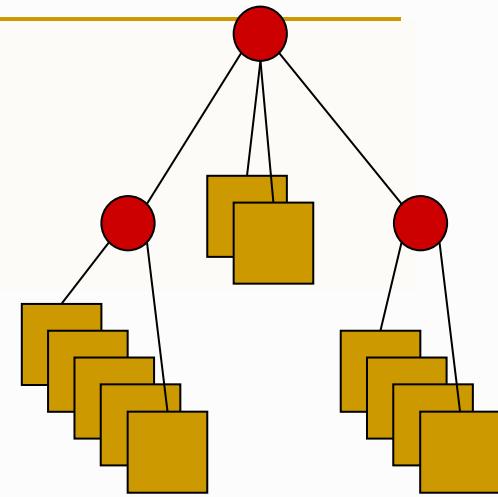


File System: objects



- ▷ Hierarchical structure for storing content
 - ♦ Provide a method for representing mount points, directories, files and links
- ▷ Mount Point
 - ♦ An access to the root of a specific FS
 - ♦ Windows uses letters (A:, .. C:..)
 - ♦ Linux, macOS, Android use any directory
- ▷ Links
 - ♦ Indirection mechanisms in a FS
 - ♦ Soft Links: point to another feature in any FS
 - Windows: Shortcuts are similar to Soft Links, but handled at the application level
 - ♦ Hard Links: provide multiple identifiers (names) for the same content (data) in the same FS
 - Usually allowed only for files
- ▷ Directory (or folder)
 - ♦ A hierarchical organization method
 - Similar to a container
 - ♦ Can contain other directories, files, mount points, links
 - ♦ The first (or top-most) is called by root
 - UNIX / Linux: '/'
 - Windows: '\'

File System: files



- ▷ Serve to store data on a persistent way
 - ♦ But longevity is given by physical support and not by the file concept ...
 - ♦ Erasing often means marked as deleted

- ▷ Ordered sequences of bytes associated with a name
 - ♦ The name allows you to retrieve/reuse these bytes later
 - ♦ Its contents can be changed, removed or added
 - As well as the name
 - ♦ They have a protection that controls their use
 - Read, write, run, remove, lock, etc. permissions
 - The protection model depends on the file system

File System: security mechanisms

▷ Mandatory protection mechanisms

- ◆ Owner
- ◆ Users and Groups allowed
- ◆ Permissions: Read, Write, Execute (run)
 - Different meanings for Files and Directories

▷ Discretionary protection mechanisms

- ◆ User-defined specific rules

▷ Additional mechanisms

- ◆ Implicit compression
- ◆ Indirection to remote resources (e.g., for OneDrive)
- ◆ Signature
- ◆ Encryption

	File	Directory
r	read contents	read contents (list entries)
w	change contents	change entries (add, rename or delete an entry)
x	execute contents	path traversal allowed

Communication Channels

- ▷ Allow the exchange of data between distinct but cooperative activities
- ▷ Essential in any current system
 - ♦ Most applications use these mechanisms
- ▷ Processes of the same SO/machine
 - ♦ Pipes, UNIX Sockets, streams, etc.
 - ♦ Communication between processes and kernel: syscalls, sockets
- ▷ Processes on different machines
 - ♦ TCP/IP and UDP/IP sockets

Access Control

- ▷ An OS kernel is an access control monitor
 - ◆ Controls all interactions with the hardware
 - ◆ Applications NEVER directly access kernel-provided resources
 - ◆ Controls all interactions between subjects and objects of its computational model

- ▷ Subjects
 - ◆ Typically, local processes (or their threads)
 - Through the system calls API
 - A syscall is not an ordinary call to a function
 - ◆ But also, messages from other machines

```
#include <stdlib.h>
#include <stdio.h>
#include <string.h>
```

```
int main()
{
    FILE * fp = fopen( "hello.txt", "wb" );
    char * str = "hello world";
    fwrite( str, strlen(str), 1, fp );
    fclose( fp );
}
```

```
$ gcc -o main ./main
```

```
$ strace ./main
```

```
....
```

```
openat(AT_FDCWD, "hello.txt", O_WRONLY|O_CREAT|O_TRUNC, 0666) = 3
```

```
fstat(3, {st_mode=S_IFREG|0644, st_size=0, ...}) = 0
```

```
write(3, "hello world", 11)          = 11
```

```
close(3)                          = 0
```

```
...
```

File interactions are mediated by the kernel
Applications do not directly access resources

Mandatory Access Control

- ▷ There are numerous cases of mandatory access control on an operating system
 - ◆ They are part of the logic of the computational model
 - ◆ They are not moldable by users and administrators
 - Unless they change the behavior of the kernel
- ▷ Examples on Linux
 - ◆ root can do everything
 - ◆ Signals to processes can only be sent by root or the owner
 - ◆ Sockets AF_PACKET (RAW) can only be created by root or by processes with the CAP_NET_RAW
- ▷ Examples on macOS
 - ◆ root can do almost anything
 - ◆ root cannot change binaries and directories signed by Apple

Discretionary Access Control

- ▷ Users can set rules for access control
 - ◆ May be definable only by the owner/user
 - This limitation is itself a Mandatory Access rule
- ▷ Examples
 - ◆ Discretionary Access Control Lists (ACL)
 - Expressive lists that limit access to resources Linux
 - ◆ Linux Apparmor
 - Stores settings in /etc/apparmor.d with application limitations
 - Rules applied automatically to applications regardless of user
 - ◆ macOS sandboxd
 - Applications are launched within isolated contexts (sandbox)
 - The sandbox contains a definition of the information that enters/exits

Protection with ACLs

- ▷ Each object has an Access Control List (ACL)
 - ◆ Tell me who can do what
- ▷ The ACL may be discretionary or mandatory
 - ◆ When it is mandatory you cannot change
 - ◆ When it is discretionary it can be changed
- ▷ It is checked when an activity intends to manipulate the object
 - ◆ If the manipulation request is not authorized it is denied
 - ◆ The SO kernel makes the ACL validations
 - Acts as a Reference Monitor

Unix file protection ACLs:

Fixed-structure, discretionary ACL

- ▷ Each file system object has an ACL
 - ◆ Binding 3 rights to 3 subjects
 - ◆ Only the owner can update the ACL
- ▷ Rights: **R W X**
 - ◆ Read / listing
 - ◆ Write / create, rename or remove files or subdirectories
 - ◆ Execute / path transversal allowance
- ▷ Subjects
 - ◆ An UID (owner)
 - ◆ A GID
 - ◆ Others (unnamed)

	UID	GID	others
	rWX	r-X	r--
octal encoding	7	5	4

```
[nobody@host ~]$ ls -la
total 12
drwxr-xr-x  2 root root 100 dez  7 21:39 .
drwxrwxrwt 25 root root 980 dez  7 21:39 ..
-rw-r----- 1 root root  6 dez  7 21:42 a
-rw-r--r--  1 root root  6 dez  7 21:42 b
-rw-r-x---+ 1 root root  6 dez  7 21:42 c
```

```
[nobody@host ~]$ cat a
cat: a: Permission denied
```

```
[nobody@host ~]$ cat b
SIO_B
```

```
[nobody@host ~]$ cat c
SIO_C
```

```
[nobody@host ~]$ getfacl c
# file: c
# owner: root
# group: root
user::rw-
user:nobody:r-x
group::r--
mask::r-x
other::---
```

Windows file protection ACLs:

Flexible-structure, discretionary ACL

▷ Each file system object has an ACL and an owner

- ♦ The ACL grants 14 types of access rights to a variable-size list of subjects
- ♦ Owner can be an UID or a GID
- ♦ Owner has no special rights over the ACL

▷ Subjects:

- ♦ Users (UIDs)
- ♦ Groups (GIDs)
 - Group “Everyone” for anybody

▷ Rights:

- ♦ Traverse Folder / Execute File
- ♦ List Folder / Read Data
- ♦ Read Attributes
- ♦ Read Extended Attributes
- ♦ Create Files /Write Data
- ♦ Create Folders / Append Data
- ♦ Write Attributes
- ♦ Write Extended Attributes
- ♦ Delete Subfolders and Files
- ♦ Delete
- ♦ Read Permissions
- ♦ Change Permissions
- ♦ Take Ownership

Privilege Elevation: Set-UID

- ▷ Changes the UID of a process running a program in a **Set-UID file**
 - ♦ In general, a process that executes a program inherits its UID from the creator process
 - Through fork + exec syscalls
 - ♦ But if
 - the program file is owned by UID X and
 - the set-UID ACL bit is set,
 - then
 - it will be executed in a process with UID X
- ▷ Used to provide privilege elevation to administration programs that are invoked by normal, untrusted users
 - ♦ Change the user's password (passwd)
 - ♦ Change to super-user mode (su, sudo)
 - ♦ Mount devices (mount)

Privilege Elevation: Set-UID

▷ Effective UID / Real UID

- ♦ Real UID is the UID of the process creator
 - App launcher
- ♦ Effective UID is the UID of the process
 - The one that really matters for defining the rights of the process

▷ UID change

- ♦ Ordinary application
 - eUID = rUID = UID of process that executed exec
 - eUID cannot be changed (unless = 0)
- ♦ Set-UID application
 - eUID = UID of exec'd application file, rUID = initial process UID
 - eUID can revert to rUID
- ♦ rUID cannot change

Privilege Elevation: Set-UID

- ▷ Administration by root is not advised
 - ◆ One “identity”, many people
 - ◆ Who did what?
- ▷ Preferable approach
 - ◆ Administration role (uid = 0), many users assume it
 - Sudoers
 - Defined by a configuration file used by sudo
- ▷ sudo is a Set-UID application with UID = 0
 - ◆ Appropriate logging can take place on each command run with sudo

```
[user@linux ~]$ ls -la /usr/sbin/sudo  
-rwsr-xr-x 1 root root 140576 nov 23 15:04 /usr/sbin/sudo
```

```
[user@linux ~]$ id  
uid=1000(user) gid=1000(user) groups=1000(user),998(sudoers)
```

```
[user@linux ~]$ sudo -s  
[sudo] password for user:
```

```
[root@linux ~]# id  
uid=0(root) gid=0(root) groups=0(root)
```

```
[root@linux ~]# exit
```

```
[user@linux ~]$ sudo id  
uid=0(root) gid=0(root) groups=0(root)
```

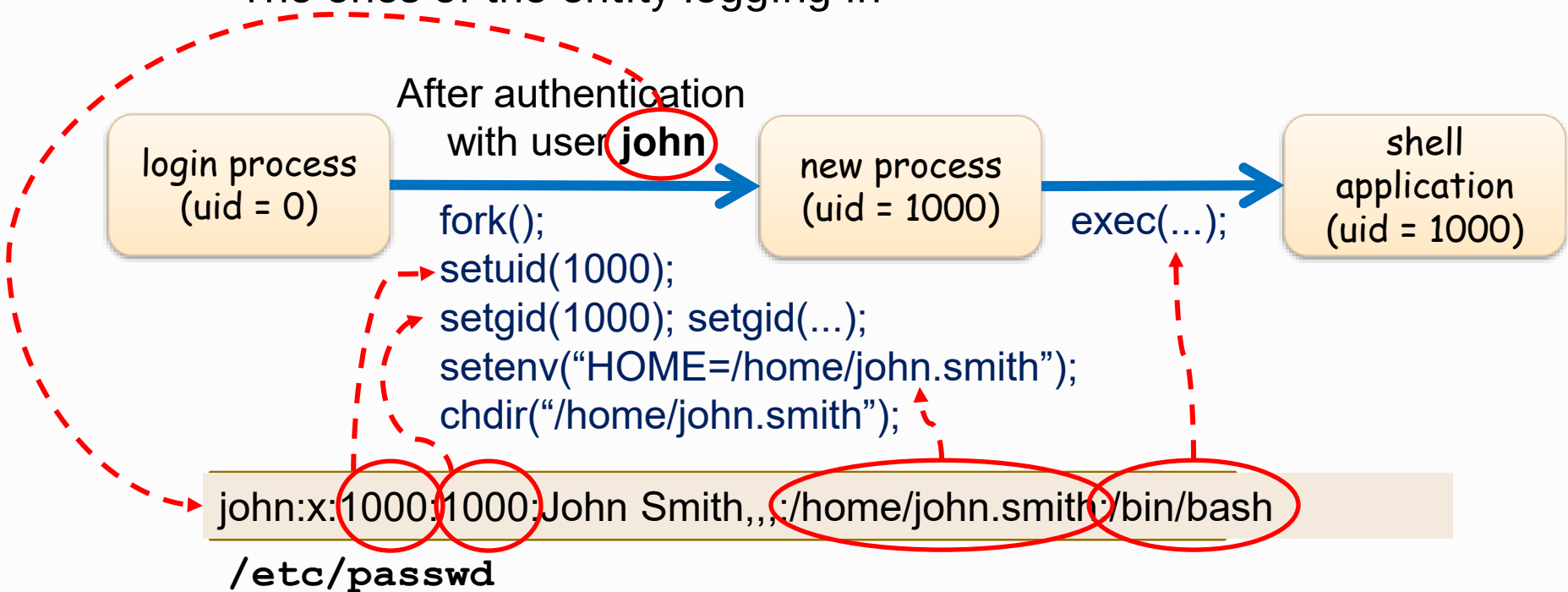
Linux login:

Not an OS kernel operation

- ▷ A privileged login application presents a login interface for getting users' credentials
 - ♦ A username/password pair
 - ♦ Biometric data
 - ♦ Smartcard and activation PIN
 - ♦ etc.
- ▷ The login application validates the credentials and fetches the appropriate UID and GIDs for the user
 - ♦ And starts an initial user application on a process with those identifiers
 - In a Linux console, this application is a shell
 - In a Linux GUI, this application is a Window Manager
 - ♦ When this process ends the login application reappears
- ▷ Thereafter all processes created by the user have its identifiers
 - ♦ Inherited through forks

Linux: from login to session processes

- ▶ The login process must be a privileged process
 - ♦ Has to create processes with arbitrary UID and GIDs
 - The ones of the entity logging in



Login in Linux:

Password validation process

- ▷ Username is used to fetch a UID + GID pair from `/etc/passwd`
 - ◆ And a set of additional GIDs from the `/etc/group` file
- ▷ Supplied password is transformed using a one-way function
 - ◆ Currently configurable, for creating a new user (`/etc/login.conf`)
 - ◆ Its identification is stored along with the transformed password
- ▷ The result is checked against a value stored in `/etc/shadow`
 - ◆ Indexed again by the `username`
 - ◆ If they match, the user was correctly authenticated
- ▷ File protections
 - ◆ `/etc/passwd` and `/etc/group` can be read by anyone
 - Required for UID/GID → user name / group name translations
 - ◆ `/etc/shadow` can only be read by root
 - Protection against dictionary attacks

Chroot mechanism

- ▷ Used to reduce the visibility of a file system
 - ◆ Each process descriptor has a root i-node number
 - From which absolute pathname resolution takes place
 - Absolute pathnames start with a “/”
 - ◆ chroot changes it to an arbitrary directory
 - The process' file system view gets reduced

- ▷ Used to protect the file system from potentially problematic applications
 - ◆ e.g., public servers, downloaded applications
 - ◆ But it is not bullet proof!

```
[root@linux /opt/chroot]# find .
```

```
.  
./usr  
./usr/lib  
./usr/lib/libcap.so.2  
./usr/lib/libreadline.so.7  
./usr/lib/libncursesw.so.6  
./usr/lib/libdl.so.2  
./usr/lib/libc.so.6  
./lib64  
./lib64/ld-linux-x86-64.so.2  
./bin  
./bin/ls  
./bin/bash
```

```
[root@linux /opt/chroot]# chroot . /bin/bash
```

```
bash-4.4# ls /
```

```
bin lib64 usr
```

```
bash-4.4# cp /bin/bash .
```

```
bash: cp: command not found
```

Confinement: AppArmor

- ▷ Mechanism for restricting applications based on a behavior model
 - ◆ Requires kernel support
 - Linux Security Modules
 - ◆ Focus on syscalls and their arguments
 - ◆ Can work in complain and enforcement modes
 - ◆ Generates entries in the system registry to audit the behavior

- ▷ Configuration files define allowed activities
 - ◆ By application, uploaded from a file
 - ◆ Applications can never have more accesses than those defined
 - Even if executed by root

Confinement: Namespaces

- ▷ Allows partitioning of resources in views (namespaces)
 - ◆ Processes in a namespace have a restricted view of the system
 - ◆ Activated through syscalls by a normal process
 - clone: Defines a namespace to migrate the process to
 - unshare: disassociates the process from its current context
 - setns: puts the process in a namespace

- ▷ Types of Namespaces
 - ◆ Mount: Applied to mount points
 - ◆ process id: first process has id 1
 - ◆ network: "independent" network stack (routes, interfaces...)
 - ◆ IPC: methods of communication between processes
 - ◆ uts: name independence (DNS)
 - ◆ user id: segregation of permissions

 - ◆ cgroup: limitation of resources used (memory, cpu...)

Confinement: Containers

- ▷ Explores namespaces to provide a virtual view of the system
 - ♦ Network isolation, user ids, mounts, cgroups, etc...
- ▷ Processes are executed inside a container
 - ♦ A container is an applicational construction and not a kernel object
 - ♦ Consists of an environment by composition of namespaces and cgroups
 - ♦ Requires building bridges with the real system network interfaces, proxy processes
- ▷ Relevant approaches
 - ♦ LXC (Linux Containers): focus on a complete virtualized environment
 - Evolution of OpenVZ
 - ♦ Docker: focus on running isolated applications based on a portable packet between systems
 - Uses LXC
 - ♦ Singularity: similar to docker, focus on HPC and multi-user sharing